

Practical Concurrent and Parallel Programming IV

Shared Memory II

Raúl Pardo

Assignment workload



- We would like to get an estimation on the number of hours you spend on assignments
- Please go to the following mentimeter poll on the right

You should indicate the number of hours that you spent to complete Assignment 1

That is, the number of hours that you spent on PCPP exercises in the last two weeks combined

Also, please write whether you submitted solutions to challenging exercises



- Why do we need a memory model?
- Basic definitions
- Correctness in the Java memory model
- Visibility
- Volatile

Chapter 5

The Java Memory Model

This chapter summarizes the Java memory model and adapts it to the purposes of the PCPP course. The changes simply omit details on how the Java memory model must be enforced by implementations of the Java Virtual Machine. The definitions and reasoning described in this note are a consistent subset of the complete Java memory model¹ [10]. Therefore, the concepts and reasoning discussed here are applicable to analyze real Java software.

The Java memory model defines, given a concurrent Java program, the set of valid ex-



- Definitions of thread-safety
 - Classes
 - Programs
- Safe publication
- Immutability
- Synchronization primitives (synchronizers)
 - Semaphores
 - Barriers
- Producer-consumer problem
- Instance confinement



*A (concurrent) program is correct if
it satisfies its specification*



- A *specification* (or *spec*) is a rigorous statement that describes the expected/desired behaviour of a program
- Examples
 - Many readers can access the shared resource at the same time, and only one can write—if no readers are reading
 - The output of the program must be `PEOPLE*num_threads`
- Specifications can be as precise as formulae in some logic (propositional, temporal, first-order, etc.)
 - We will not cover these details in the course

Reasoning about concurrent programs



- We have covered the basic concepts to reason about the *correctness* of concurrent programs
- Reasoning about correctness of concurrent programs is tricky
 - You have experienced this already in the assignments where you work with programs consisting in a few lines of code
- Imagine having to reason about applications with thousands of lines of code and many classes
 - Server applications
 - Operating Systems
 - GUIs
 - ...



Modular class-based reasoning



- It is more manageable to separately analyse parts of the code and then combine them in safe ways
- In Object Oriented languages (such as Java) we can focus on analysing thread-safety for classes
- This reduces the analysis to concurrent method calls and field accesses



- A ***data race*** occurs when two concurrent threads:
 - Access a shared memory location
 - At least one access is a write
 - The accesses are not ordered by the happens-before relation

Conflicting
accesses

A class is thread-safe if no concurrent execution of method calls or field accesses (read/write) on objects of this class results in data races on the fields of the class

PCPP teaching team

Thread-safe class

· 12



Inspired by the Java memory model: “A program is correctly synchronized iff none of its executions contains data races”

A class is thread-safe if no concurrent execution of method calls or field accesses (read/write) on objects of this class results in data races on the fields of the class

Note that this definition is independent of class invariants as opposed to Goetz Chapter 4. This definition is more similar to Goetz Chapter 2, page 18.

PCPP teaching team

Thread-safe class

Inspired by the Java memory model: “A program is correctly synchronized iff none of its executions contains data races”

IMPORTANT: In this course, *thread-safety* is not an umbrella term for code that seem to behave correctly in concurrent environments.



· 12

A class is thread-safe if no concurrent execution of method calls or field accesses (read/write) on objects of this class results in data races on the fields of the class

Note that this definition is independent of class invariants as opposed to Goetz Chapter 4. This definition is more similar to Goetz Chapter 2, page 18.

PCPP teaching team



IMPORTANT: In this course, *thread-safety* is not an umbrella term for code that seem to behave correctly in concurrent environments.

memory model: “A
synchronized iff none
contains data races”

What is the specification in this definition?

A **class** is **thread-safe** if no concurrent execution of method calls or field accesses (read/write) on objects of this class results in data races on the fields of the class

Note that this definition is independent of class invariants as opposed to Goetz Chapter 4. This definition is more similar to Goetz Chapter 2, page 18.

PCPP teaching team

Thread-safe program



Do not confuse thread-safe classes with thread-safe programs.
Thread-safe programs are not defined in Goetz.

*A concurrent program is thread-safe
if and only if it is data race free*

Inspired by *correctly synchronized
programs* in the Java memory model

PCPP teaching team



- To analyse whether a class is thread-safe, we must simply ensure that for any concurrent execution of field access and methods calls—where at least one write access is executed—the actions are related by the happens-before relation
- In what follows, we list the elements to identify/ensure:
 - Class state
 - Escaping
 - Safe publication
 - Immutability
 - Mutual exclusion



- To analyse whether a class is thread-safe, we must simply ensure that for any concurrent execution of field access and methods calls—where at least one write access is executed—the actions are related by the happens-before relation
- In what follows, we list the elements to identify/ensure:
 - Class state
 - Escaping
 - Safe publication
 - Immutability
 - Mutual exclusion

When asked to reason about the thread-safety of a class, you must always cover these elements



- Recall that uncontrolled concurrent access to the shared variables may lead to data races
- The state of a class includes the fields defined in the class
 - Our goal is to ensure that concurrent access to class state is free from data races
- The fields of the class are shared variables for a class that is to be used concurrently

```
class C {  
    // class state (variables)  
    T s1;  
    T s2;  
    T s3;  
    T s4;  
    ...  
  
    // class methods  
    T m1 (...) {...}  
    T m2 (...) {...}  
    T m3 (...) {...}  
    ...  
}
```

Only class state



- Our definition of thread-safe class focuses on data races on the fields of the class
 - Data races on fields that are not part of class state do not violate the definition

```
class C {  
    // class state (variables)  
    private int i = 0;  
  
    // class methods  
    public void synchronized m(List<Integer> l) {  
        l.add(42);  
    }  
}
```



- Our definition of thread-safe class focuses on data races on the fields of the class
 - Data races on fields that are not part of class state do not violate the definition
- Methods should avoid using object references as parameters
 - We cannot guarantee happens-before order on actions involving the referenced object
- The data race on the right does not violate thread safety

```
class C {  
    // class state (variables)  
    private int i = 0;  
  
    // class methods  
    public void synchronized m(List<Integer> l) {  
        l.add(42);  
    }  
}
```

```
// program using C  
  
List<Integer> l = new ArrayList<Integer>();  
C c = new C();  
new Thread(() -> {  
    l.add(1);  
}).start();  
  
new Thread(() -> {  
    c.m(l); // the operations between the two  
           // threads are not related by  
           // happens-before  
}).start();
```



```
class Counter {  
    // class state (variables)  
    int i=0;  
  
    // class methods  
    public synchronized void inc(){i++;}  
}
```



- Class state does not escape if it can only be accessed through public methods
- It is important to not expose class fields
- Otherwise, threads may use them without ensuring proper synchronization
 - Thus, we cannot enforce that the operations are related by happens-before

```
class Counter {  
    // class state (variables)  
    int i=0;  
  
    // class methods  
    public synchronized void inc(){i++;}  
}
```

```
// program using Counter  
  
Counter c = new Counter();  
new Thread(() -> {  
    c.inc();  
}).start();  
  
new Thread(() -> {  
    c.i++; // escaped the lock in inc()  
}).start();
```



- Class state does not escape if it can only be accessed through public methods
- It is important to not expose class fields
- Otherwise, threads may use them without ensuring proper synchronization
 - Thus, we cannot enforce that the operations are related by happens-before
- Defining all (primitive) class fields as private ensures that these variables will only be accessed through public methods.
 - Thus, it is easier to control and reason about concurrent access

```
class Counter {  
    // class state (variables)  
    int i=0;  
  
    // class methods  
    public synchronized void inc(){i++;}  
}
```

```
// program using Counter  
  
Counter c = new Counter();  
new Thread(() -> {  
    c.inc();  
}).start();  
  
new Thread(() -> {  
    c.i++; // escaped the lock in inc()  
}).start();
```



```
class IntArrayList {  
    // class state  
    private List<Integer> a = new ArrayList<Integer>();  
  
    public synchronized void set(Integer index, Integer elem)  
    { a.set(index,elem); }  
  
    public synchronized List<Integer> get() { return a; }  
}
```



Can list a escape in IntArrayList?



menti.com

8844 4584

```
class IntArrayList {  
    // class state  
    private List<Integer> a = new ArrayList<Integer>();  
  
    public synchronized void set(Integer index, Integer elem)  
    { a.set(index,elem); }  
  
    public synchronized List<Integer> get() { return a; }  
}
```



Can list a escape in IntArrayList?



menti.com

8844 4584

```
class IntArrayList {  
    // class state  
    private List<Integer> a = new ArrayList<Integer>();  
  
    public synchronized void set(Integer index, Integer elem)  
    { a.set(index,elem); }  
  
    public synchronized List<Integer> get() { return a; }  
}
```

```
IntArrayList array = new IntArrayList();  
new Thread() -> {  
    array.set(0,1); // access state with lock  
}).start();  
new Thread() -> {  
    array.get().set(0,42); // access state without locks  
}).start();
```



- Remember that when a method returns an object, we get a *reference* to that object
- Therefore, even if we obtain the reference using locks, later we can modify the content of the object without locks
- To prevent escaping, avoid methods that return references to objects in the class state

```
class IntArrayList {  
    // class state  
    private List<Integer> a = new ArrayList<Integer>();  
  
    public synchronized void set(Integer index, Integer elem)  
    { a.set(index,elem); }  
  
    public synchronized List<Integer> get() { return a; }  
}
```

```
IntArrayList array = new IntArrayList();  
new Thread(() -> {  
    array.set(0,1); // access state with lock  
}).start();  
new Thread(() -> {  
    array.get().set(0,42); // access state without locks  
}).start();
```



- Class state is safely published if, after initializing an object of the class, the initial values of the fields of the class are visible to any thread accessing the object.
- Safe publication requires that initialization *happens-before* publication



- Class state is safely published if, after initializing an object of the class, the initial values of the fields of the class are visible to any thread accessing the object.
- Safe publication requires that initialization *happens-before* publication
- Visibility issues may appear during initialization of objects

```
public class UnsafeInitialization {  
    private int x;  
    private Object o;  
    public UnsafeInitialization() {  
        x = 42;  
        o = new Object();  
    }  
}
```



- Class state is safely published if, after initializing an object of the class, the initial values of the fields of the class are visible to any thread accessing the object.
- Safe publication requires that initialization *happens-before* publication
- Visibility issues may appear during initialization of objects

```
public class UnsafeInitialization {  
    private int x;  
    private Object o;  
    public UnsafeInitialization() {  
        x = 42;  
        o = new Object();  
    }  
}
```

- For the thread that executed the constructor, there are no visibility issues, but if a reference to an instance of UnsafeInitialization object is accessible to another thread, it might not see **x==42** or o completely initialized

If the lack of visibility is due to leaking the reference before the constructor finished execution, then it is not a safe publication issue. That said, you must never leak a reference to the object in the middle of constructor.



- Class state is safely published if, after initializing an object of the class, the initial values of the fields of the class are visible to any thread accessing the object.
- Safe publication requires that initialization *happens-before* publication
- Visibility issues may appear during initialization of objects

```
public class UnsafeInitialization {  
    private int x;  
    private Object o;  
    public UnsafeInitialization() {  
        x = 42;  
        o = new Object();  
    }  
}
```

- For the thread that executed the constructor, there are no visibility issues, but if a reference to an instance of UnsafeInitialization object is accessible to another thread, it might not see **x==42** or **o** completely initialized



- We can address visibility issues during initialization as follows

```
public class UnsafeInitialization {  
    private int x;  
    private final Object o;  
    public UnsafeInitialization() {  
        x = 0;  
        o = new Object();  
    }  
}
```



- We can address visibility issues during initialization as follows

For primitive types, we can:

- Initialize them as the default value. For `int` it is `0`. (only works if the default value is acceptable)
- Declare them as **`volatile`** (only if the field is intended to be volatile, otherwise it introduces a performance penalty)
- Declare them as **`final`** (only works if the content is never modified)
- Declare them as **`static`** (only works if the field must be **`static`** in the class)

```
public class UnsafeInitialization {  
    private int x;  
    private final Object o;  
    public UnsafeInitialization() {  
        x = 0;  
        o = new Object();  
    }  
}
```



- We can address visibility issues during initialization as follows

For primitive types, we can:

- Initialize them as the default value. For int it is 0. (only works if the default value is acceptable)
- Declare them as **volatile** (only if the field is intended to be volatile, otherwise it introduces a performance penalty)
- Declare them as **final** (only works if the content is never modified)
- Declare them as **static** (only works if the field must be **static** in the class)

```
public class UnsafeInitialization {  
    private int x;  
    private final Object o;  
    public UnsafeInitialization() {  
        x = 0;  
        o = new Object();  
    }  
}
```

For reference type objects, we can:

- Declare them as **final** Initialize(this is the standard way, as only the reference to the object the which they point becomes immutable)
- Initialize them as the default value: null. (only works if the default value is acceptable)
- Declare them as **static** (only works if the field must be **static** in the class)



- The previous suggestions ensure safe publication because:
 - The actions for initialization and access to the object's reference (publication) are ordered by *happens-before*
 - *A write to a volatile field happens-before every subsequent read of that field.*
 - *The default initialization (zero, false, or null) of any object happens-before any other actions of a program.*
 - *The initialization of final and static fields happens-before any other actions of a program (after the constructor has finished its execution)*
 - At the JVM level, the reason is that
 - The initial value of **final** and **static** fields cannot remain in cache after the constructor finishes
 - All fields are initialized with default values during class loading
 - Writes on **volatile** are flushed to main memory (during initialization)



- The previous suggestions ensure safe publication because:
 - The actions for initialization and access to the object's reference (publication) are ordered by *happens-before*
 - *A write to a volatile field happens-before every subsequent read of that field.*
 - *The default initialization (zero, false, or null) of any object happens-before any other actions of a program.*
 - *The initialization of final and static fields happens-before any other actions of a program (after the constructor has finished its execution)*
 - At the JVM level, the reason is that
 - The initial value of **final** and **static** fields cannot remain in cache after the constructor finishes
 - All fields are initialized with default values during class loading
 - Writes on **volatile** are flushed to main memory (during initialization)

Defined by us from the JLS explanation and Goetz. You can use it for exercises in this course.

NOTE: For clarity and simplicity, up to now, we did not take initialization concerns into account. But from now on we will.

- The previous suggestions ensure safe publication because:
 - The actions for initialization and access to the object's reference (publication) are ordered by *happens-before*
 - *A write to a volatile field happens-before every subsequent read of that field.*
 - *The default initialization (zero, false, or null) of any object happens-before any other actions of a program.*
 - *The initialization of final and static fields happens-before any other actions of a program (after the constructor has finished its execution)*
 - At the JVM level, the reason is that
 - The initial value of **final** and **static** fields cannot remain in cache after the constructor finishes
 - All fields are initialized with default values during class loading
 - Writes on **volatile** are flushed to main memory (during initialization)

Defined by us from the JLS explanation and Goetz. You can use it for exercises in this course.

- An immutable field is one whose value does not change after initialization
 - You can think of it as a constant
 - The **final** keyword in Java prevents modification of fields
 - Remember that variables assigned to an object only hold a reference to the object
- Since immutable fields do not change the state after initialization, data races can only occur during initialization

- To ensure thread-safety of immutable fields, we must ensure that:
 - The field cannot be modified after publication
 - The field is safely published
 - The field does not escape



- To ensure thread-safety of immutable fields, we must ensure that:
 - The field cannot be modified after publication
 - The field is safely published
 - The field does not escape

```
public final class ThreeStooges {  
    private final Set<String> stooges = new HashSet<String>();  
  
    public ThreeStooges () {  
        stooges.add("Moe");  
        stooges.add("Larry");  
        stooges.add("Curly");  
    }  
  
    public Boolean isStooge(String name) {  
        return stooges.contains(name)  
    }  
}
```

Goetz p. 47

Why don't we need to ensure mutual exclusion in isStooge(...) to guarantee thread-safety?

able fields, we must ensure

- The field cannot be modified after publication
- The field is safely published
- The field does not escape



menti.com
8844 4584

```
public final class ThreeStooges {  
    private final Set<String> stooges = new HashSet<String>();  
  
    public ThreeStooges () {  
        stooges.add("Moe");  
        stooges.add("Larry");  
        stooges.add("Curly");  
    }  
  
    public Boolean isStooge(String name) {  
        return stooges.contains(name)  
    }  
}
```

Goetz p. 47

Mutual exclusion



· 44

- If we have mutable class state, then we must ensure mutual exclusion



- If we have mutable class state, then we must ensure mutual exclusion

```
public final class Stooges {  
    private final Set<String> stooges = new HashSet<String>();  
  
    public ThreeStooges () {  
        stooges.add("Moe");  
        stooges.add("Larry");  
        stooges.add("Curly");  
    }  
  
    public synchronized Boolean isStooge(String name) {  
        return stooges.contains(name)  
    }  
  
    public synchronized void addStooge(String name) {  
        stooges.add(name);  
    }  
}
```

This write access make **stooges** mutable



- To analyse thread-safe in a class, we must identify/consider:
 - Identify the class state
 - Make sure that class state does not escape
 - Ensure safe publication
 - Whenever possible define class state as immutable
 - If class state must be mutable, ensure mutual exclusion

Interesting section (4.5) on documenting synchronization in Goetz. Unfortunately, not widespread.

Thread-safe classes

Theorem (see optional readings): These properties are sufficient to ensure the thread-safety of a class—defined as data-race freedom for any pair of field accesses in any concurrent execution of method calls and field accesses

· 47



- To analyse thread-safe in a class, we must identify/consider:
 - Identify the class state
 - Make sure that class state does not escape
 - Ensure safe publication
 - Whenever possible define class state as immutable
 - If class state must be mutable, ensure mutual exclusion

Interesting section (4.5) on documenting synchronization in Goetz. Unfortunately, not widespread.

Other synchronization primitives (synchronizers)



- Semaphores are synchronization primitives that allow c number of threads in the critical section where c is called the *capacity*
 - First introduced by Dijkstra
- A semaphore consists of:
 - An integer capacity (c), [permits in Java](#)
 - Initial number of threads allowed in the critical section
 - A method **acquire()**
 - Checks if $c > 0$, if so, it decrements capacity by one ($c--$) and allows the calling thread to make progress, otherwise it blocks the thread
 - It is a blocking call
 - A method **release()**
 - It checks whether there are waiting threads, if so, it awakes one of them, otherwise it increases the capacity by one ($c++$)
 - It is non-blocking



- Semaphores are synchronization primitives that allow c number of threads in the critical section where c is called the *capacity*
 - First introduced by Dijkstra
- A semaphore consists of:
 - An integer capacity (c), [permits in Java](#)
 - Initial number of threads allowed in the critical section
 - A method **acquire()**
 - Checks if $c > 0$, if so, it decrements capacity by one ($c--$) and allows the calling thread to make progress, otherwise it blocks the thread
 - It is a blocking call
 - A method **release()**
 - It checks whether there are waiting threads, if so, it awakes one of them, otherwise it increases the capacity by one ($c++$)
 - It is non-blocking

Semaphores (1968) appear
before Monitors (1972)



Synchronization primitives that only allow one thread in the critical section are called **mutex** (which is short for mutual exclusion)

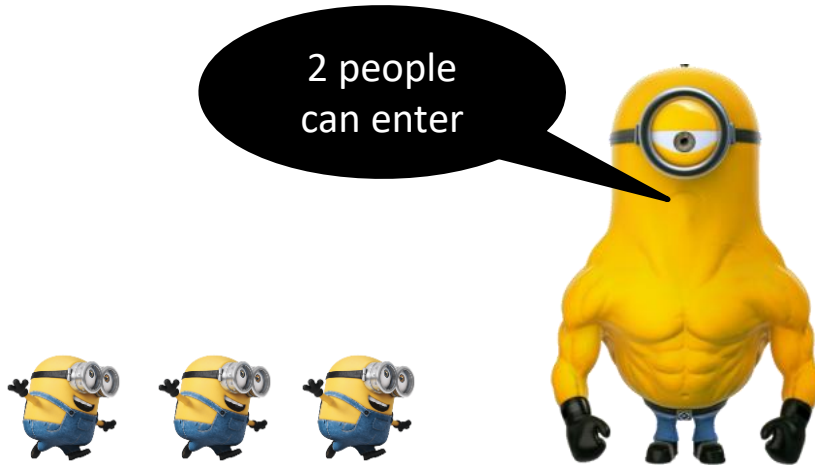
Synchronization primitives that allow c number of threads in the critical section are called **semaphores**, where c is called the *capacity*

Semaphores (1968) appear before Monitors (1972)

- A semaphore consists of:
 - An integer capacity (c), [permits in Java](#)
 - Initial number of threads allowed in the critical section
 - A method **acquire()**
 - Checks if $c > 0$, if so, it decrements capacity by one ($c--$) and allows the calling thread to make progress, otherwise it blocks the thread
 - It is a blocking call
 - A method **release()**
 - It checks whether there are waiting threads, if so, it awakes one of them, otherwise it increases the capacity by one ($c++$)
 - It is non-blocking



- You can think of a semaphore as a “bouncer” to enter a critical section or to be allowed to use a shared resource





- You can think of a semaphore as a “bouncer” to enter a critical section or to be allowed to use a shared resource





- You can think of a semaphore as a “bouncer” to enter a critical section or to be allowed to use a shared resource



- You can think of a semaphore as a “bouncer” to enter a critical section or to be allowed to use a shared resource





- You can think of a semaphore as a “bouncer” to enter a critical section or to be allowed to use a shared resource





- Semaphores are typically used to control the number of threads accessing a resource (here we fix a maximum 5 readers)

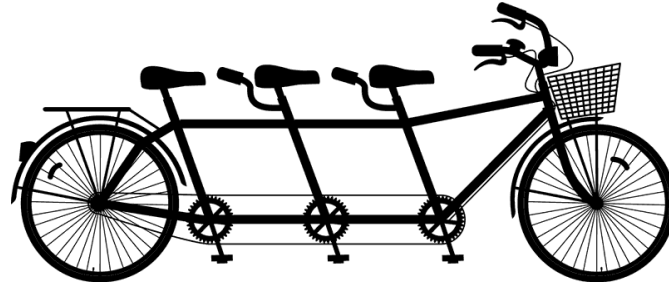
```
ReadWriteMonitor m = new ReadWriteMonitor();  
Semaphore semReaders = new Semaphore(5, true);  
for (int i = 0; i < 10; i++) {  
    // start a reader  
    new Thread() -> {  
        m.readLock();  
        semReaders.acquire();  
        // read  
        semReaders.release();  
        m.readUnlock();  
    }.start();  
  
    // start a writer  
    new Thread() -> {  
        m.writeLock();  
        // write  
        m.writeUnlock();  
    }.start();  
}
```

Java semaphores have a fair flag so that their entry queue prioritizes the longest waiting thread

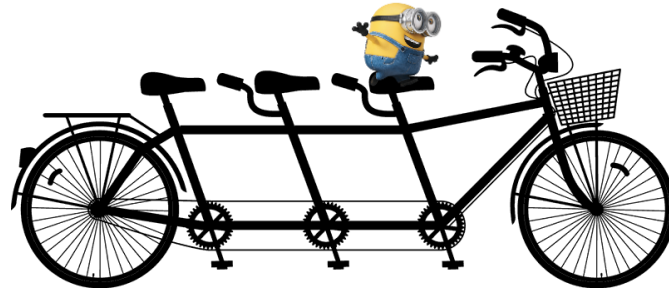
See `ReadersWritersSemaphore.java`

- *Barriers* are synchronization primitives used to wait until several threads reach some point in their computation

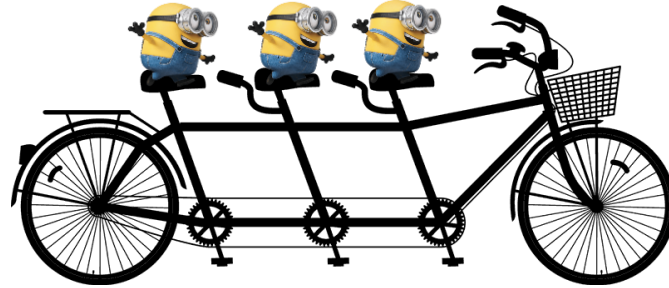
- *Barriers* are synchronization primitives used to wait until several thread reach some point in their computation



- *Barriers* are synchronization primitives used to wait until several thread reach some point in their computation



- *Barriers* are synchronization primitives used to wait until several thread reach some point in their computation





- *Barriers* are synchronization primitives used to wait until several thread reach some point in their computation
- Barriers consists of
 - A number *parties* to wait for
 - A method **`await()`**
 - If the number of waiting threads is less than *parties*, then the calling thread blocks, otherwise all waiting threads awake and the calling thread is allowed to make progress
- Java includes the class **`CyclicBarrier`**
 - After *parties* called **`await()`**, then the state is reset and the barrier behaves as initially

Barrier Example | Parallel initialization



· 65

- Several threads are used to initialize an array (each a different position), the barrier is used for threads to know when the initialization is finished
 - This example is a bit artificial, but it illustrates the use of barriers.

```
int parties          = 10;
CyclicBarrier cb     = new CyclicBarrier(parties);
int[] shared_array = new int[parties];
...
for (int i = 0; i < parties; i++) {
    new SetterClass(i).start();
}
...
public class SetterClass extends Thread {
    int index;
    public SetterClass(int index) {this.index = index;}

    public void run() {
        shared_array[index] = index+1;
        cb.await();
        // After this point the array is initialized and it is safe to read it
    }
}
```

Barrier Example | Parallel initialization



· 65

- Several threads are used to initialize an array (each a different position), the barrier is used for threads to know when the initialization is finished
 - This example is a bit artificial, but it illustrates the use of barriers.

```
int parties          = 10;
CyclicBarrier cb     = new CyclicBarrier(parties);
int[] shared_array = new int[parties];
...
for (int i = 0; i < parties; i++) {
    new SetterClass(i).start();
}
...
public class SetterClass extends Thread {
    int index;
    public SetterClass(int index) {this.index = index;}

    public void run() {
        shared_array[index] = index+1;
        cb.await();
        // After this point the array is initialized and it is safe to read it
    }
}
```

See `BarrierExample.java`

Producer-consumer problem



- Consider a shared data structure of fixed size from which threads may add and remove elements
- Producer threads may add elements to the structure as long as it is not full
 - If the structure is full and a producer tries to add an element, it must block until there an element is removed
- Consumer threads remove elements to the structure as long as it is not empty
 - If the structure is empty and a consumer tries to remove an element, then it must block until an element is added
- A good solution to the problem must be deadlock free and minimize starvation

Producer-consumer problem | Intuition



· 67

- Perhaps more intuitive example

Consumers

Producers



Shared data structure of fixed size

- The producer-consumer problem appears in many multi-threaded situations
 - Handling access to a shared bounded data structure
 - Controlling access to limited computational resources
 - E.g., thread pools
 - Asynchronous I/O operations
 - External devices may act as producers providing data to the system (keyboard, mouse, etc...), or consumer obtaining tasks to perform (IoT devices)

Instance confinement

Instance confinement



- *Instance confinement* refers to encapsulating access to a thread-unsafe object into a thread-safe class



- *Instance confinement* refers to encapsulating access to a thread-unsafe object into a thread-safe class

```
public class PersonSet {  
    private final Set<Person> mySet = new HashSet<Person>();  
  
    public synchronized void addPerson (Person p) {  
        mySet.add(p);  
    }  
  
    public synchronized boolean contains(Person p) {  
        return mySet.contains(p);  
    }  
}
```

Goetz p. 59

- *Instance confinement* refers to encapsulating access to a thread-unsafe object into a thread-safe class

```
public class PersonSet {  
    private final Set<Person> mySet = new HashSet<Person>();  
  
    public synchronized void addPerson (Person p) {  
        mySet.add(p);  
    }  
  
    public synchronized boolean contains(Person p) {  
        return mySet.contains(p);  
    }  
}
```

Goetz p. 59

Java synchronized collections is an example of instance confinement. Any Java collection can be turned into a thread-safe collection via the `synchronized<collection>` method, e.g., `synchronized List` : ([https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Collections.html#synchronizedList\(java.util.List\)](https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Collections.html#synchronizedList(java.util.List)))

p only accesses thread-safe classes $\nRightarrow$ absence of race conditions

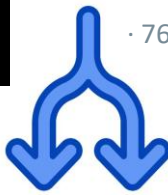


· 76

p only accesses thread-safe classes $\Rightarrow$ absence of race conditions



```
List<Integer> l = new ArrayList<Integer>();  
List<Integer> lSync = Collections.synchronizedList(l);  
  
...  
  
new Thread(() -> { addIfAbsent(lSync,l); }).start();  
new Thread(() -> { addIfAbsent(lSync,l); }).start();  
  
...  
  
public void addIfAbsent(List l, Integer e) {  
    if (!l.contains(e))  
        l.add(e);  
}
```



Is this program race condition free?

```
List<Integer> l = new ArrayList<Integer>();
List<Integer> lSync = Collections.synchronizedList(l);

...

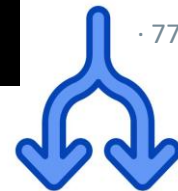
new Thread(() -> { addIfAbsent(lSync,1); }).start();
new Thread(() -> { addIfAbsent(lSync,1); }).start();

...

public void addIfAbsent(List l, Integer e) {
    if (!l.contains(e))
        l.add(e);
}
```



menti.com
8844 4584



Is this program race condition free?

The program is data race free, as all accesses are ordered by happens-before

```
List<Integer> l = new ArrayList<Integer>();  
List<Integer> lSync = Collections.synchronizedList(l);  
  
...  
  
new Thread(() -> { addIfAbsent(lSync,1); }).start();  
new Thread(() -> { addIfAbsent(lSync,1); }).start();  
  
...  
  
public void addIfAbsent(List l, Integer e) {  
    if (!l.contains(e))  
        l.add(e);  
}
```



- Thread-safe classes may be extended to include compound actions
 - Intuitively, compound actions can be seen multiple method calls or field accesses within a critical section
 - Common examples are: *check-and-set*, iteration, navigation (*contains*)

```
public void addIfAbsent(List l, Integer e) {  
    synchronized (l) {  
        if (!l.contains(e))  
            l.add(e);  
    }  
}
```

Thread uses the intrinsic lock of a thread-safe collection

```
class ThreadSafeList {  
    ...  
    public void synchronized addIfAbsent(T e) {  
        if (!l.contains(e))  
            l.add(e);  
    }  
    ...  
}
```

Thread-safe class is extended with a custom method to perform the action

- Definitions of thread-safety
 - Classes
 - Programs
- Safe publication
- Immutability
- Synchronization primitives (synchronizers)
 - Semaphores
 - Barriers
- Producer-consumer problem
- Instance confinement